Vectorized Neural Network

Alan Mathew Joby
CSE, RSET
u2503036@rajagiri.edu.in

Abstract

This document provides the mathematical foundation and architectural implementation details for a fully connected, single-hidden-layer neural network engineered entirely from first principles using Python and NumPy. The system features a custom computer vision pipeline designed to ingest, process, and accurately classify raw, real-world handwritten inputs. The core engine is structured explicitly for structural data integrity and optimized matrix manipulations.

1 Architectural Overview

This document outlines the operational metrics of a custom computer vision classification layout structured entirely around the following baseline parameters:

- **Input Topology:** 64 Features (Flat 8×8 Grayscale Image Vectors)
- **Hidden Topology:** 128 Neurons with Rectified Linear Unit (ReLU) Activation
- **Output Topology:** 10 Latent Categories via a Numerically Stable Softmax Layer
- **Loss Profile:** Multi-Class Cross-Entropy

2 Mathematical Framework & Vectorization

Unlike traditional scalar loops, this engine operates completely on vectorized batches to optimize system memory access and cache layout frameworks.

2.1 Forward Propagation

For a batch input matrix $X \in \mathbb{R}^{M \times 64}$:

$$Z_1 = XW_1 + b_1 \tag{1}$$

$$A_1 = \max(0, Z_1) \tag{2}$$

$$Z_2 = A_1W_2 + b_2 \tag{3}$$

$$A_2 = \text{Softmax}(Z_2) \tag{4}$$

Where the Softmax activation is structurally modified for numerical stability against floating-point bit limits by isolating the maximum scaling factor:

$$\text{Softmax}(Z)_{i,j} = \frac{e^{Z_{i,j} - \max(Z_{i,:})}}{\sum_k e^{Z_{i,k} - \max(Z_{i,:})}} \tag{5}$$

2.2 Vectorized Backpropagation

The matrix gradients tracking the total loss error across dimensional transformations are explicitly mapped without relying on automated tracking configurations.

Output Layer Error Tracking Matrix:

$$dZ_2 = \frac{A_2 - Y}{M} \tag{6}$$

Hidden-to-Output Parameter Gradients:

$$dW_2 = A_1^T \cdot dZ_2, \quad db_2 = \sum_{\text{axis}=0} dZ_2 \tag{7}$$

Hidden Layer Activation Masking:

$$dA_1 = dZ_2 \cdot W_2^T \tag{8}$$

$$dZ_1 = dA_1 \odot \mathbb{I}(Z_1 > 0) \tag{9}$$

Input-to-Hidden Parameter Gradients:

$$dW_1 = X^T \cdot dZ_1, \quad db_1 = \sum_{\text{axis}=0} dZ_1 \tag{10}$$

3 Advanced Computer Vision Domain Adaptation

Deploying models on uncurated, real-world camera images requires a strict preprocessing pipeline to correct for structural noise, illumination, and scale variances.

```
[Raw Photo Input]

[Grayscale Conversion] -------------> Drops chroma variance noise

[Dynamic Bounding Box Tracking] ----> Locates exact coordinate boundaries via mask (.getbbox())

[Polarity Inversion] ---------------> Swaps values to match training domain (bright on dark)

[Aspect-Ratio Uniform Padding] -----> Creates a uniform square matrix without distorting shape

[Lanczos Downsampling] -------------> Samples image down to the explicit 8x8 footprint
```

Figure 1: *Pipeline visualization for raw image structural domain adaptation.*

4 System Stability & Performance Logs

During execution over 1,000 iterations at a uniform learning rate ($\alpha = 0.1$), the core engine displayed optimal parameter updates before hitting standard optimization boundaries.

Initial State: Custom image classification yields 47.77% confidence, displaying close distribution overlaps with visually similar numeric structures (e.g., classes 1 and 9).

Convergence Behavior: Cross-Entropy loss smoothly settled down from initial levels to 0.0309, matching a near-perfect training dataset representation of 99.9%.

Generalization Limits: The system reached its absolute ceiling at 97.22% test accuracy. Beyond this point, the metrics entered a slight oscillation loop, indicating standard parameter overfitting and learning-rate overshooting across the loss valley.

Post-Convergence Performance: The custom image test showed a massive confidence increase, locking onto a highly sharp 75.79% positive target identification.